1. Setiap aktivasi dinormasilasi memakai rata-rata dan varians dari mini-batch yang sedang dilatih, lalu diskalakan dengan γ dan β. Karena *mini-batch* merupakan sampel acak, statistik tersebut hanyalah estimasi yang berisik dari statistik populasi, sehingga nilai ternormalisasi suatu citra berubah bergantung pada citra lain dalam *batch*-nya (Ioffe & Szegedy, 201). Gangguan acak ini bekerja mirip *noise* pada *Dropout* dan mencegah jaringan bergantung pada nilai aktivasi yang persis, sehingga efeknya berupa regularisasi ringan (Slide 8). Saat inferensi, BN memakai *running average* yang tetap sehingga *noise* ini hilang.

   Penghilangan *Dropout* berlaku ketika *batch size* terlalu kecil atau terlalu besar. *Batch size* yang terlalu kecil, statistik terlalu berisik sehingga pelatihan tidak stabil. Pada *batch size* yang terlalu besar, *noise* mengecil dan regularisasinya melemah. Selain itu, *Dropout* sebelum lapisan BN dapat menggeser varians antara pelatihan dan inferensi sehingga akurasi turun (Li et al., 2019).

2. Dalam CNN, penambahan lapisan dapat menurunkan akurasi karena semakin dalam lapisan CNN, semakin sulit untuk dioptimasi (He et al., 2016).

   Skip connection mengubah *output* blok menjadi $y = F(x) + x$ (Slide 11). Pada *backpropagation*, turunannya menjadi $\delta y/\delta x = \delta F/\delta x + 1$, sehingga gradien memiliki jalur langsung yang tidak melewati lapisan berbobot. Untuk menjadi identitas, lapisan cukup membuat $F(x)$ mendekati nol, yang jauh lebih mudah dipelajari.

   Vanishing gradient terjadi karena gradien dikalikan turunan yang kecil berulang kali, sehingga mengecil menuju nol pada lapisan awal. Suku +1 menjaga gradien tetap mengalir ke lapisan awal, sehingga jaringan sangat dalam seperti ResNet (Slide 12) tetap dapat dilatih.

3. Jumlah parameter yang diperbarui:
   - *Feature extraction*: Pendekatan ini membekukan seluruh lapisan *backbone* dan hanya mengaktifkan lapisan pengklasifikasi. Jumlah parameter sedikit.
   - *Fine-tuning*: Pendekatan ini mengaktifkan sebagian kecil sampai seluruh lapisan dalam model. Jumlah parameter jauh lebih banyak.

   Kebutuhan volume data latih:
   - *Feature extraction*: Karena jumlah parameter sedikit, maka volume data yang dibutuhkan juga sedikit.
   - *Fine-tuning*: Membutuhkan volume data latih yang jauh lebih besar. Karena pendekatan ini menyesuaikan lebih banyak bobot dalam arsitektur, ketersediaan data yang besar diperlukan untuk mencegah model mengalami *overfitting*.

   Beban komputasi:
   - *Feature extraction*: Beban komputasi sangat ringan karena jumlah proses *backpropagation* yang dilakukan tidak banyak.

- *Fine-tuning*: Beban komputasi tinggi karena sebagian atau seluruh lapisan dari model membutuhkan proses *backpropagation* yang memerlukan banyak memori.

Risiko rusaknya fitur representasi bawaan:

- *Feature extraction*: Tidak ada risiko rusaknya fitur bawaan karena seluruh bobot representasi pada lapisan awal dikunci.
- *Fine-tuning*: Memiliki risiko yang sangat nyata berupa hilangnya pengetahuan awal jika proses *fine-tuning* dilakukan terlalu cepat atau salah penanganan. Untuk menekan risiko kerusakan fitur ini, *fine-tuning* wajib menggunakan nilai pembelajaran (*learning rate*) yang lebih rendah.

Praktik yang tepat: Pada awal *training*, selalu mulai dengan *feature extraction* agar dapat menstabilkan bobot acak pada *classifier*. Jika akurasi belum sesuai, maka *fine-tuning* dapat dilakukan untuk melakukan penyesuaian khusus.

4. Augmentasi hanya diaktifkan pada *train set* untuk mensimulasikan data pelatihan tambahan guna meningkatkan generalisasi model (Slide 20). Pada *validation/test set*, augmentasi wajib dinonaktifkan agar evaluasi secara objektif mengukur performa model pada distribusi citra dunia nyata yang asli tanpa distorsi buatan.

   Contoh transformasi yang tidak layak adalah Color Jitter ekstrem (Slide 21). Syarat utama augmentasi adalah mempertahankan invariansi semantik (identitas kelas tidak berubah). Pada ras hewan, warna bulu sering menjadi fitur pembeda krusial (contohnya: *Golden Retriever* berwarna kuning vs *Flat-Coated Retriever* berwarna hitam). Mengubah *hue* warna secara drastis akan merusak semantik label asli tersebut, sehingga model justru mempelajari representasi fitur yang salah.

5. Fenomena ini terjadi ketika gradien error yang besar dari tugas baru langsung menimpa bobot bawaan secara drastis (Slide 28). Hal ini umumnya dipicu karena lapisan *classifier* baru masih memiliki bobot acak. Jika langsung dilatih bersamaan, *backpropagation* akan menghasilkan pembaruan agresif yang merusak representasi fitur universal (seperti pola tepi/tekstur) dari data awal.

   Strategi:

   - Membuka pembekuan secara bertahap dari belakang, agar *classifier* mencapai stabilitas gradien terlebih dahulu. Hal ini dapat mencegah propagasi gradien acak yang besar merusak lapisan dasar yang sudah stabil.
   - Mengatur *learning rate* lebih rendah saat *fine-tuning*. Hal ini memastikan penyesuaian terjadi secara stabil tanpa menghapus memori representasi bawaan.

Daftar pustaka:

Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. In Proceedings of the 32nd International Conference on Machine Learning (Vol. 37, pp. 448–456).

Li, X., Chen, S., Hu, X., & Yang, J. (2019). Understanding the disharmony between dropout and batch normalization by variance shift. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (pp. 2682–2690).

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) (pp. 770–778).